SURPLUS FOOD RESCUE & MARKETPLACE

A project report
submitted in fulfillment of
the requirements for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

From
APJ Abdul Kalam Kerala Technological University

Submitted By

RISHIN V (MEA25MCA2021)

MEA Engineering College
Department of Computer Applications
Vengoor P.O, Perinthalmanna, Malappuram, Kerala-679325

OCTOBER 2026

DEPARTMENT OF COMPUTER APPLICATIONS
MEA ENGINEERING COLLEGE
PERINTHALMANNA-679325

CERTIFICATE

This is to certify that the Project report entitled **"Surplus Food Rescue & Marketplace"** is a bonafide record of the work done by **RISHIN V (MEA25MCA2021)** under our supervision and guidance. The report has been submitted in fulfillment of the requirement for award of the Degree of Master of Computer Applications from the APJ Abdul Kalam Kerala Technological University for the year 2026.

Ms. Fathmath Hasna
Assistant Professor
Project Guide
Dept. of Computer Applications

Mr. Sajeesh M
Assistant Professor & Head
Head of the Department
Dept. of Computer Applications

ACKNOWLEDGEMENTS

First and foremost, I would like to thank Almighty God for giving me the knowledge, courage, and strength which helped me in the successful completion of this project.

An endeavor of this magnitude can only be successful with the advice, motivation, and guidance of many well-wishers. I take this opportunity to express my heartfelt gratitude to all who encouraged me throughout this journey. I express my deep sense of gratitude to our respected Principal, **Dr. Shahir VK**, for his inspiring leadership and for fostering an excellent academic environment in the college.

I would like to express my deepest gratitude and sincere indebtedness to **Mr. Sajeesh M**, Assistant Professor and Head of the Department, Department of Computer Applications, for providing permission and facilities to conduct this project in a systematic way.

I am profoundly thankful to my Project Guide, **Ms. Fathmath Hasna**, Assistant Professor in the Department of Computer Applications, for her invaluable technical guidance, timely advice, constructive critique, and wholehearted moral support throughout the conception, development, and completion of the Surplus Food Rescue & Marketplace project.

I also extend my sincere gratitude to all the esteemed faculty members and technical staff of the Department of Computer Applications for their constant cooperation, encouragement, and valuable assistance during various stages of this project.

RISHIN V (MEA25MCA2021)

DATE: OCTOBER 25, 2026

ABSTRACT

Food wastage represents a staggering ecological and socio-economic challenge worldwide. Massive quantities of perfectly edible, wholesome cooked dishes, artisanal bakery goods, and grocery surplus are discarded daily by commercial food businesses simply because items remain unsold at the conclusion of regular trading hours. Concurrently, consumers encounter steep inflation on food commodities, while local charities and price-conscious individuals struggle to secure nutritious sustenance affordably.

To effectively solve this systemic inefficiency, **Surplus Food Rescue & Marketplace (EcoBite)** is engineered as a robust, real-time web-based marketplace dedicated to bridging food business surplus directly with neighborhood consumers. The platform provides dedicated, role-based access for three distinct operational actors: **Customer**, **Restaurant Partner**, and **Platform Administrator**.

Customers can discover certified nearby bakeries and eateries, filter surplus food items by cuisine, pricing, and allergen restrictions, buy curated discounted surplus bundles or "Daily Mystery Bags" at 50% to 80% below retail value, securely checkout, and receive instant cryptographic OTP pickup tokens alongside scannable QR verification codes. Restaurant Partners can easily list excess inventory with strict collection timeframes, monitor incoming customer bookings in real time via WebSocket events, and validate customer redemptions using built-in QR camera scanning or PIN confirmation. Platform Administrators retain comprehensive oversight by conducting credential verification of registered restaurants, auditing food catalogs, executing automated cleanup sweeps of expired foodstuffs, and monitoring system-wide financial and environmental savings metrics.

The platform is developed using the modern MERN Stack (React.js, Node.js, Express.js, and MongoDB Atlas) paired with Socket.io for instantaneous multi-party notifications, JWT for role-based session authorization, Bcrypt for credential encryption, and automated background Node-Cron tasks for continuous auto-expiry and dynamic price markdown orchestration. The application delivers an efficient, reliable, and sustainable solution to urban food wastage, establishing a complete enterprise-grade full-stack web application suitable for academic excellence and real-world commercial deployment.

CONTENTS

1.	**INTRODUCTION**	**1**
2.	**SYSTEM ANALYSIS**	**3**
	2.1 Existing System	4
	2.2 Proposed System	4
	2.3 Module Description	4
	2.4 Sprint	5
3.	**FEASIBILITY STUDY**	**8**
	3.1 Economical Feasibility	9
	3.2 Technical Feasibility	9
	3.3 Operational Feasibility	9
	3.4 Behavioural Feasibility	9
	3.5 Software Feasibility	10
	3.6 Hardware Feasibility	10
4.	**SOFTWARE ENGINEERING PARADIGM**	**11**
	4.1 Agile Model	12
	4.2 Scrum	12
5.	**SYSTEM REQUIREMENT SPECIFICATION**	**13**
	5.1 Software Requirements	14
	5.2 Hardware Requirements	14
6.	**SYSTEM DESIGN**	**15**
	6.1 Database Design	16
	6.2 Tables	18
	6.3 UML Design	20
	6.4 Use Case Diagram	21
	6.5 Scenario	22
	6.6 Data flow diagram	23
7.	**SYSTEM DEVELOPMENT**	**27**
	7.1 Coding	28
8.	**SYSTEM TESTING AND IMPLEMENTATION**	**31**

8.1 Types of Testing 32

8.2 Implementation 34

9. SYSTEM MAINTENANCE 35

10. FUTURE ENHANCEMENT 37

11. CONCLUSION 39

12. APPENDIX 41

13. BIBLIOGRAPHY 48

INTRODUCTION

1. INTRODUCTION

Food loss and culinary waste have emerged as severe ecological, economic, and moral challenges of contemporary society. Each year, billions of tons of high-grade, wholesome cooked dishes, artisan breads, delicate pastries, and grocery items are produced and transported only to be dumped into municipal landfills. Within commercial food establishments—including restaurants, bakeries, cafes, and supermarket hot-bars—food items that remain unpurchased at the end of the operating shift are routinely discarded due to the perishable nature of fresh items, stringent aesthetic guidelines, and lack of dynamic redistribution channels.

This widespread wastage generates severe environmental consequences. Decomposing organic food products in open landfills produce massive quantities of methane, a potent greenhouse gas that accelerates global climatic warming. Concurrently, consumers and working families face sharp cost-of-living increases, making nutritious meals expensive to afford regularly. Traditional food redistribution approaches, such as sporadic charitable donations or word-of-mouth discounts, are fundamentally uncoordinated, slow, and operationally burdensome for commercial kitchen managers.

To address these critical inefficiencies, I developed the **Surplus Food Rescue & Marketplace (EcoBite)** platform. The system establishes a synchronized, hyper-local digital marketplace connecting food businesses burdened with surplus stock directly with conscientious neighborhood consumers looking for discounted meals. The application provides dedicated role-based portals for Customers, Restaurant Partners, and System Administrators, engineered to eliminate operational delays and facilitate friction-free transactions.

The platform empowers customers to discover verified local food stores, filter items by diet, cuisine, and allergen sensitivities, order mystery surprise bags at steep 50% to 80% markdowns, and retrieve food securely using tamper-proof OTP collection codes and QR passes. Restaurants gain an intuitive merchant hub to monetize excess stock, broadcast dynamic discounts during final trading minutes, and verify collections effortlessly with camera-based QR scanning. Administrators oversee platform health, verify restaurant licenses, audit food listings, and monitor aggregated carbon offsets.

Engineered on the modern MERN Stack (React.js, Node.js, Express.js, and MongoDB Atlas) with Socket.io real-time websockets, the Surplus Food platform provides an efficient, transparent, and scalable solution to urban food wastage, perfectly suited for academic evaluation and real-world deployment.

SYSTEM ANALYSIS

2. SYSTEM ANALYSIS

2.1 Existing System

In the existing food retail ecosystem, restaurants, grocery vendors, and bakeries rely heavily on manual forecasting to calculate daily preparation quantities. Because consumer demand fluctuates wildly depending on weather, local foot traffic, and daily events, kitchens consistently prepare surplus inventory to prevent running out of menu offerings. At the conclusion of daily trading, leftover prepared food is routinely discarded into trash compactors.

Currently, there is no unified digital channel enabling merchants to rapidly alert nearby customers of available surplus items during closing hours. Traditional food bank donations, while noble, require scheduled logistical coordination, refrigerated transport vehicles, and extensive paperwork, making them impractical for small, hot perishable batches. Furthermore, consumers lack a trusted mechanism to identify which local stores have discounted items available for immediate collection, leading to massive financial waste for businesses and preventable environmental damage.

2.2 Proposed System

The proposed system, **Surplus Food Rescue & Marketplace**, introduces a centralized full-stack web application designed to eliminate food waste by facilitating rapid, hyper-local surplus food transactions. The system introduces dedicated, role-based workflows for Customers, Restaurant Partners, and Administrators.

Customers can browse available food listings in real time, filter by dietary requirements (Veg, Non-Veg, Vegan, Gluten-Free, Halal), reserve curated Mystery Surprise Bags, and receive a secure 6-digit OTP token and scannable QR verification code. Restaurants can post surplus inventory in under a minute, set designated pickup windows, receive instantaneous WebSocket booking notifications, and verify redemptions seamlessly with built-in QR scanning. Platform Administrators ensure marketplace trust by auditing restaurant credentials, monitoring active postings, and analyzing system-wide sustainability metrics.

2.3 Module Description

The Surplus Food platform is structured into three specialized functional modules based on user roles, each tailored to execute distinct business workflows:

Admin Module

- Secure administrator authentication and multi-tier role-based access control.
- Audit, approve, or decline onboarding restaurant business profiles and operating credentials.
- Comprehensive registry management to monitor all registered customers and merchant accounts.

- System-wide inventory monitoring across all active surplus food postings.
- Automated maintenance trigger to sweep and delete expired food items past pickup windows.
- Live analytics dashboard displaying total meals rescued, financial customer savings, and CO2 emissions prevented.

Customer Module

- User registration, secure email OTP validation, and JWT session authentication.
- Interactive food exploration hub with filtering by cuisine, price range, dietary preferences, and allergens.
- Surplus Surprise Bag discovery offering mystery assortments at 50% to 80% markdowns.
- Real-time booking and checkout with automated collection token (OTP) and QR code generation.
- Digital reservation voucher download in printable PDF receipt format.
- Personalized reservations tracker displaying active pickup windows and past order histories.
- Post-pickup review and star rating submission to maintain vendor quality.

Restaurant Module

- Merchant onboarding with business name, address, cuisine specialization, and logo uploads.
- Surplus food item listing engine with original price, discounted price, stock count, and strict pickup hours.
- Surplus Mystery Bag toggle with pre-configured value estimates and descriptions.
- Real-time WebSocket booking alerts notifying kitchen staff immediately upon customer order placement.
- Customer collection verification via manual 6-digit OTP entry or built-in camera QR scanner.
- Live merchant analytics tracking total sales revenue, units rescued, and customer satisfaction ratings.

2.4 Sprint

Sprint 1: Core Authentication & Customer Portal

Module	Task	Pending task if any	Hour of completion	Expected date of completion	Actual date of completion	Reason for delay
Customer	User Registration & Email OTP Verification	-	2 hr.	19/07/2026	19/07/2026	-
	JWT Login & Role Authorization	-	2 hr.	21/07/2026	21/07/2026	-
	Surplus Catalog & Filter Grid (Cuisine/Allergens)	-	4 hr.	23/07/2026	23/07/2026	-
	Food Reservation & OTP Token Engine	-	3 hr.	25/07/2026	25/07/2026	-
	QR Code Pass & PDF Receipt Generation	-	4 hr.	28/07/2026	28/07/2026	-

Sprint 2: Restaurant Hub & Real-time Verification

Module	Task	Pending task if any	Hour of completion	Expected date of completion	Actual date of completion	Reason for delay
Restaurant	Store Profile Setup & Verification Submission	-	2 hr.	10/08/2026	10/08/2026	-
	Surplus Listing Management & Image Uploads	-	5 hr.	12/08/2026	12/08/2026	-
	Surprise Bag & Dynamic	-	3 hr.	14/08/2026	14/08/2026	-

Module	Task	Pending task if any	Hour of completion	Expected date of completion	Actual date of completion	Reason for delay
	Markdown Pricing					
	WebSocket Real-Time Order Dispatch	-	4 hr.	15/08/2026	15/08/2026	-
	Camera QR Scanner & Token Verification	-	3 hr.	17/08/2026	17/08/2026	-

Sprint 3: Administration, Automated Jobs & Analytics

Module	Task	Pending task if any	Hour of completion	Expected date of completion	Actual date of completion	Reason for delay
Admin	Admin Command Center & Access Controls	-	2 hr.	18/08/2026	18/08/2026	-
	Merchant Store Profile Verification & Auditing	-	4 hr.	20/08/2026	20/08/2026	-
	Global Postings Monitor & Expired Sweeper	-	4 hr.	22/08/2026	22/08/2026	-
	Node-Cron Auto-Expiry & Markdown Engine	-	5 hr.	24/08/2026	24/08/2026	-
	System Metrics, User Management & Analytics	-	3 hr.	28/08/2026	28/08/2026	-

FEASIBILITY STUDY

3. FEASIBILITY STUDY

An analysis of the ability to complete a project successfully, taking into account legal, economic, technological, scheduling, and other factors is considered as a feasibility study. Rather than just diving into a project and hoping for the best, a feasibility study allows project managers to investigate the possible negative and positive outcomes of a project before investing too much time and money.

3.1 Economical Feasibility

The proposed system is economically highly feasible because it is built entirely using robust, open-source technologies including React.js, Node.js, Express.js, MongoDB Atlas (Community Tier), Socket.io, Git, and GitHub. None of these core technologies require proprietary or expensive software licensing fees. Development and testing were executed on standard consumer computing hardware using freely available, industry-standard developer tooling such as Visual Studio Code and Postman. Since the primary objective of Surplus Food Rescue & Marketplace is food waste mitigation and academic evaluation, the capital expenditure and operational maintenance costs remain exceptionally low, while delivering an enterprise-grade commercial web application.

3.2 Technical Feasibility

The project is technically feasible as all necessary frameworks and libraries are mature, extensively documented, and supported by vibrant global developer communities. The MERN Stack provides an asynchronous, non-blocking architecture ideally suited for high-throughput marketplace communications. MongoDB provides high-performance schema flexibility for nested surplus food catalogs and order tracking. Socket.io guarantees bidirectional WebSocket dispatching with sub-second latencies, essential for alerting merchants of immediate orders. JWT ensures cryptographic, stateless role-based security, while HTML5 QR libraries enable effortless in-browser barcode verification. The system architecture has been thoroughly verified using modern full-stack software development methodologies.

3.3 Operational Feasibility

Surplus Food Rescue & Marketplace is operationally feasible because its user interfaces are deliberately designed for fast-paced food service scenarios with minimal cognitive load. For customers, the intuitive catalog allows one-click filtering, instant cartless reservations, and scannable QR passes requiring zero technical training. For restaurant staff, the responsive dashboard provides live audio-visual order alerts and one-tap camera QR scanning that integrates seamlessly into frantic kitchen closing routines. For administrators, the centralized operational dashboard provides clear tabular auditing of store registrations and item listings. This streamlined design ensures that users across all skill levels can operate the system effectively.

3.4 Behavioural Feasibility

Behavioral feasibility examines the willingness and confidence of stakeholders to adopt the proposed digital system. Restaurant owners are strongly motivated to adopt the system because it transforms what was previously discarded as pure financial loss into a supplemental revenue stream, while bolstering their public environmental reputation. Customers are enthusiastically driven to participate by substantial discounts (50% to 80% markdowns) on restaurant-quality food, combined with the altruistic satisfaction of rescuing edible food from landfills. These powerful positive behavioral drivers guarantee high user adoption and sustained community engagement.

3.5 Software Feasibility

The software requirements for the project encompass a standard modern operating system (Windows 10/11, Linux, or macOS) supporting Node.js (v18+) runtime environment, React.js (v19/18) with Vite for frontend rendering, Express.js (v4.19) for backend REST microservices, and MongoDB Atlas for distributed database storage. Ancillary software includes Socket.io for bidirectional WebSocket messaging, jsPDF and QRCode.react for client-side receipt generation, JSON Web Token (JWT) and Bcrypt for encryption and authorization, Dotenv for secure environment variables, and Git/GitHub for distributed version control. All software components are readily accessible, production-stable, and fully compatible.

3.6 Hardware Feasibility

The hardware infrastructure necessary to run and interact with Surplus Food Rescue & Marketplace is widely available and affordable. The server-side platform can be hosted on standard cloud instances or local machines with a minimum of an Intel Core i3 / AMD Ryzen dual-core processor, 8 GB of RAM, and modest solid-state storage. On the client side, the web application runs responsively on any standard desktop, laptop, or mobile smartphone with a standard web browser and an internet connection (broadband, 4G, or 5G). No specialized POS hardware or proprietary scanners are required, making the deployment highly cost-effective and scalable.

SOFTWARE ENGINEERING PARADIGM

4. SOFTWARE ENGINEERING PARADIGM

The software engineering paradigm which is also referred to as a software process model or Software Development Life Cycle (SDLC) model is the development strategy that encompasses the process, methods and tools. SDLC describes the period of time that starts with the software system being conceptualized.

4.1 Agile model

The Agile SDLC model is a combination of iterative and incremental process models with focus on process adaptability and customer satisfaction by rapid delivery of working software product. Agile Methods break the product into small incremental builds. These builds are provided in iterations. Each iteration typically lasts from about one to three weeks. At the end of the iteration, a working product is displayed to the customer and important stakeholders.

In Agile, the tasks are divided to time boxes (small time frames) to deliver specific features for a release. Iterative approach is taken and working software build is delivered after each iteration. Each build is incremental in terms of features; the final build holds all the features required by the customer.

4.2 Scrum

Scrum is an agile framework for managing knowledge work, with an emphasis on software development. It is designed for teams of three to nine members, who break their work into actions that can be completed within time boxed iterations, called "sprints", no longer than one month and most commonly two weeks, then track progress and re-plan in 15-minute stand-up meetings, called daily scrums.

Scrum is an iterative and incremental framework for product development. It promotes a flexible strategy where development teams collaborate to achieve common goals, challenging traditional sequential methods. Scrum encourages self-organization, physical co-location, and daily communication among team members.

SYSTEM REQUIREMENTS SPECIFICATION

5. SYSTEM REQUIREMENTS SPECIFICATION

5.1 Software Requirements

Component	Requirement
Operating System	Windows 10 / 11, Linux (Ubuntu 20.04+), macOS
Backend Runtime	Node.js (v18+), Express.js (v4.19), Socket.io (v4.8)
Frontend Framework	React.js (v19/18), Vite, Lucide-React, Axios
Database	MongoDB Atlas (v6.0+) / MongoDB Community Edition
Ancillary Libraries	Bcryptjs, JSONWebToken, HTML5-QRCode, jsPDF, Node-Cron
Development IDE & Tools	Visual Studio Code, Postman API Client, Git, GitHub

5.2 Hardware Requirements

Component	Requirement
Processor	Intel Core i3 / i5 or AMD Ryzen equivalent (2.0 GHz or above)
RAM	Minimum 8 GB (16 GB Recommended for development)
Storage	256 GB SSD or higher for database and runtime logs
Display Monitor	14" Full HD (1920x1080) or standard modern mobile display
Camera Peripherals	Standard webcam or smartphone camera (for QR scanning)
Network	Broadband / 4G / 5G active internet connection

SYSTEM DESIGN

6. SYSTEM DESIGN

System design is the process of architecting the overall structure, modular components, database entities, communication interfaces, and operational data flows of a software application to fulfill specified functional and performance criteria. The system design of Surplus Food Rescue & Marketplace focuses on engineering an ultra-responsive, highly available, resilient, and fault-tolerant architecture capable of coordinating time-sensitive food rescues, real-time inventory synchronization, dynamic markdown price drops, and rapid QR token validation during closing hours.

6.1 Database Design

Database design organizes and models system entities to ensure high data integrity, minimal redundancy, lightning-fast querying, and rapid transaction processing under concurrent customer ordering spikes. The Surplus Food platform utilizes MongoDB, an enterprise-grade NoSQL document-oriented database. Unlike rigid relational databases, MongoDB stores information in flexible, schema-validated JSON-like BSON documents within specialized collections. This structure natively supports polymorphic attributes such as allergen arrays, dietary classifications, dynamic price adjustments, and nested payment details.

The database architecture is designed to manage complex relationships among customer users, restaurant profiles, surplus food listings, reservations/orders, and customer reviews. Unique ObjectIds and referenced indexes maintain data consistency across all collections.

The major collections used in the system include:

- **Users** — Stores user account credentials, hashed passwords (Bcrypt), contact phone numbers, assigned roles (customer, restaurant, admin), email verification statuses, and OTP codes.
- **RestaurantProfiles** — Stores business metadata, restaurant name, street address, cuisine classification, operational description, store banner image paths, administrative verification flag, and aggregated customer ratings.
- **FoodItems** — Stores surplus inventory listings, referenced restaurant profile IDs, item titles, descriptions, original pricing, discounted offer pricing, available quantity, daily pickup time windows, expiry timestamps, dietary badges, and Surprise Bag indicators.
- **Orders** — Stores transaction records, referenced customer ID, restaurant ID, food item ID, quantity reserved, total price, order status (reserved, collected, cancelled), unique 6-digit OTP verification token, and payment details.
- **Reviews** — Stores post-pickup ratings (1 to 5 stars), written customer comments, and references linking the reviewer, store, and verified order.

6.2 Tables (Database Schemas & Data Dictionary)

Database Schema 1: Users Collection (Customers, Restaurants, Admins)

```
_id         : ObjectId('6a85e825d3f0510078fbec0c')
name        : "John Doe"
email       : "customer@surplus.com"
password    : "$2a$10$wN7KjZq0/2K3B...hashed..."
role        : "customer"
phoneNumber : "555-123-4567"
isVerified  : true
otpCode     : ""
createdAt   : 2026-08-16T10:15:30.000+00:00
__v         : 0
```

Database Schema 2: RestaurantProfiles Collection

```
_id            : ObjectId('6a85ea2eff5323a524c4f07d')
user           : ObjectId('6a85e825d3f0510078fbec0a')
restaurantName : "The Green Bakery"
address        : "123 Bread Street, Foodtown"
cuisineType    : "Bakeries"
description    : "An artisanal bakery crafting organic bread, pastries, and croissants fresh
daily."
imagePath      : "/uploads/store_banner.jpg"
isVerified     : true
rating         : 5.0
ratingCount    : 0
createdAt      : 2026-08-16T10:20:00.000+00:00
__v            : 0
```

Database Schema 3: FoodItems Collection (Surplus Listings & Mystery Bags)

```
_id                : ObjectId('6ad1ab025bc04b489f991070')
restaurant         : ObjectId('6a85ea2eff5323a524c4f07d')
name               : "Evening Bakery Surprise Bag"
description        : "A mystery assortment of fresh daily surplus baked goods and artisan so
urdough."
originalPrice      : 15.00
discountedPrice    : 4.99
quantity           : 6
pickupStartTime    : "18:00"
pickupEndTime      : "21:00"
status             : "available"
expiryDate         : 2026-10-18T21:00:00.000+00:00
dietaryType        : "veg"
allergens          : ["Nut-Free"]
isSurpriseBag      : true
estimatedValue     : 15.00
isDynamicDiscounted : false
createdAt          : 2026-10-16T10:11:38.000+00:00
__v                : 0
```

Database Schema 4: Orders Collection (Reservations & Collection Tokens)

```
_id            : ObjectId('6ad1abf55bc04b489f99108b')
customer       : ObjectId('6a85e825d3f0510078fbec0c')
restaurant     : ObjectId('6a85ea2eff5323a524c4f07d')
foodItem       : ObjectId('6ad1ab025bc04b489f991070')
quantity       : 1
totalPrice     : 2.50
status         : "reserved"
token          : "387421"
paymentStatus  : "paid"
paymentDetails : {
                   "transactionId": "TXN-16237966",
                   "cardLast4": "4242"
                 }
createdAt      : 2026-10-16T10:12:34.000+00:00
__v            : 0
```

Database Schema 5: Reviews Collection

```
_id        : ObjectId('6ad1ac225bc04b489f991094')
customer   : ObjectId('6a85e825d3f0510078fbec0c')
restaurant : ObjectId('6a85ea2eff5323a524c4f07d')
order      : ObjectId('6ad1abf55bc04b489f99108b')
rating     : 5
comment    : "Outstanding quality! Delicious croissants and bread at an unbelievable discoun
t."
createdAt  : 2026-10-16T10:15:00.000+00:00
```

```
__v       : 0
```

6.3 UML Designs

The Unified Modeling Language (UML) is a standard language for specifying, visualizing, constructing, and documenting the artifacts of software systems, as well as for business modeling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems. UML uses mostly graphical notations to express the design of software projects. Using UML helps project teams communicate, explore potential designs, and validate the architectural design of the software.

6.4 Use Case Diagram

A use case diagram is a visual representation that shows how different users (called actors) interact with a system to achieve specific goals. It is one of the primary diagrams used in UML to capture functional requirements. Each use case represents a particular function or service the system provides, and the relationships between actors and use cases show who can perform which actions.

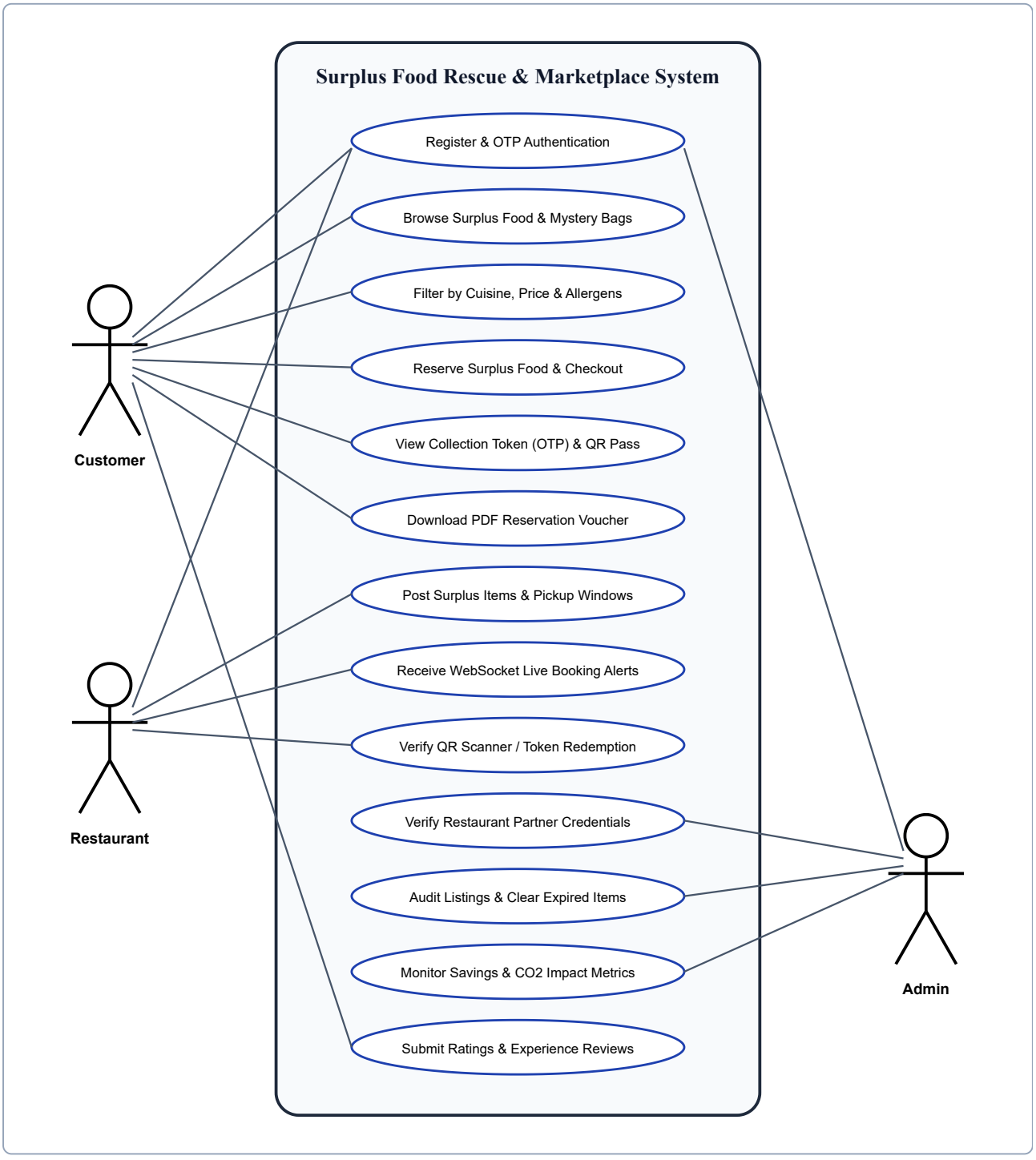


Figure 6.1: UML Use Case Diagram — Surplus Food Platform

6.5 Scenario

A scenario is a detailed description of a particular situation or sequence of events that helps illustrate how a system or process works. In software engineering, scenarios are often used to show how users interact with a system to achieve specific goals. For example, a scenario might describe how a customer discovers a local bakery, reserves a surprise bag, receives an OTP token, and collects the package. Scenarios make it easier to understand user needs, test different conditions, and visualize the flow of operations within a system.

CUSTOMER SCENARIOS

- Customer can register an account and verify identity via 6-digit email OTP.
- Customer can authenticate securely using email and password to receive a JWT session.
- Customer can explore active surplus food items from nearby verified restaurants.
- Customer can filter available offerings by cuisine classification, max price, dietary type, and allergens.
- Customer can view Mystery Surprise Bags offered at heavy markdowns.
- Customer can reserve surplus items, specify quantities, and complete simulated checkout.
- Customer can receive a unique 6-digit collection OTP and dynamic scannable QR verification code.
- Customer can download an official PDF reservation receipt with pickup instructions.
- Customer can view active reservations, store addresses, and pickup closing windows.
- Customer can submit star ratings and qualitative reviews following successful collection.
- Customer can securely log out of the platform.

RESTAURANT SCENARIOS

- Restaurant can register and submit restaurant profile credentials (name, address, cuisine) for audit.
- Restaurant can post surplus inventory with original price, discounted price, stock count, and collection times.
- Restaurant can flag listings as Surprise Mystery Bags with estimated retail valuations.
- Restaurant can update or remove active surplus listings.
- Restaurant can receive instantaneous WebSocket sound and banner alerts when customer reservations are placed.
- Restaurant can view the customer reservation queue in real time.
- Restaurant can verify customer pickups by entering the 6-digit OTP code or scanning customer QR codes with a webcam.
- Restaurant can track revenue metrics, total stock quantities, and customer satisfaction ratings.

- Restaurant can securely log out of the platform.

ADMIN SCENARIOS

- Administrator can securely log in via protected admin credentials.
- Administrator can review, approve, or reject onboarding restaurant business profiles.
- Administrator can view all registered customer and merchant user accounts.
- Administrator can inspect all active platform food listings and verify pricing integrity.
- Administrator can execute automated database cleanups to purge expired food listings.
- Administrator can track aggregate environmental and economic metrics (meals rescued, customer savings, CO2 prevented).
- Administrator can securely log out of the admin console.

6.6 Data Flow Diagram

Level 0: Context Diagram

The Level 0 Data Flow Diagram (Context Diagram) represents the entire Surplus Food Rescue & Marketplace platform as a single centralized process, showing high-level data input and output flows exchanged with external entities (Customer, Restaurant Partner, and Admin).

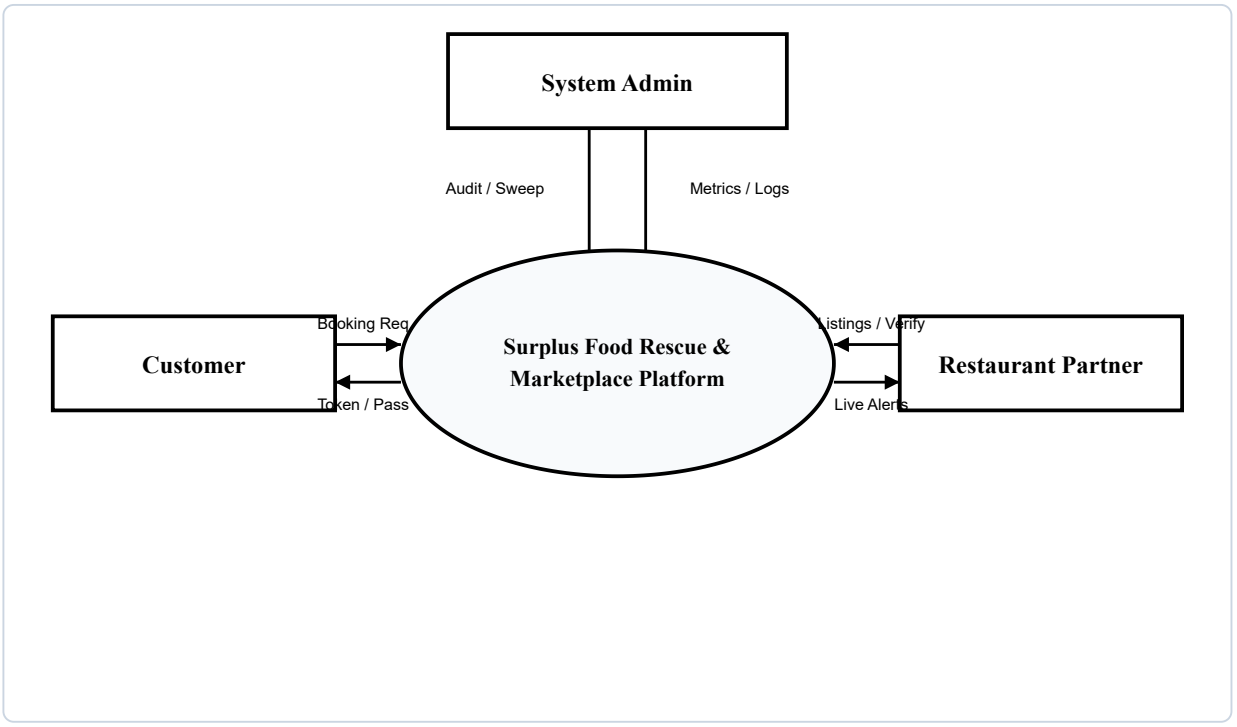
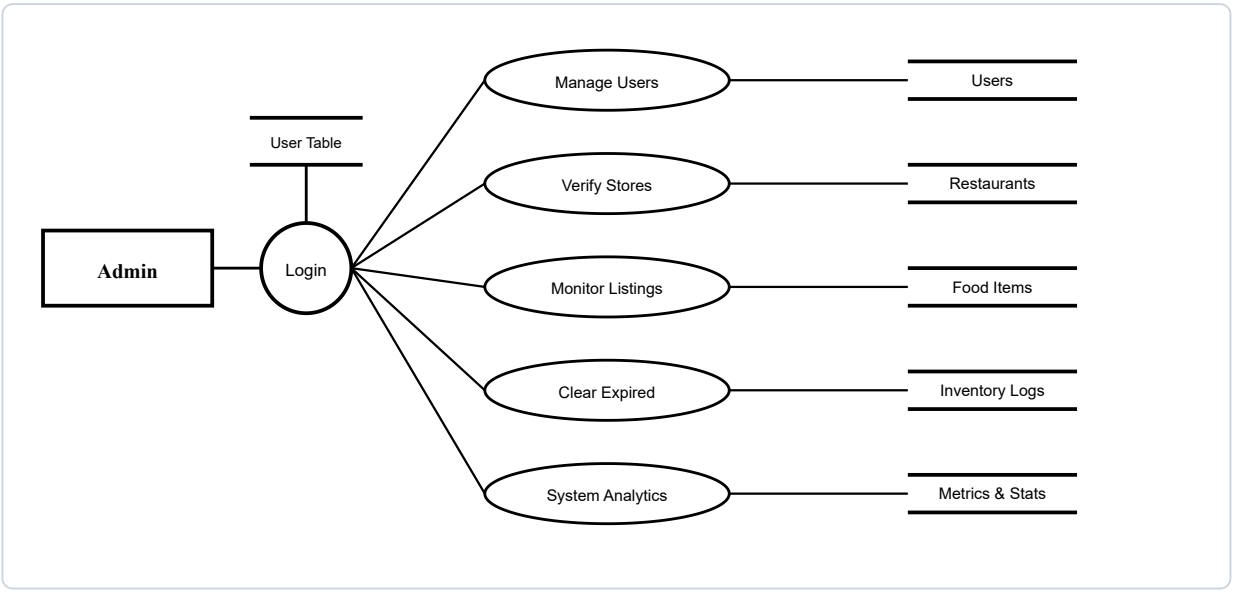


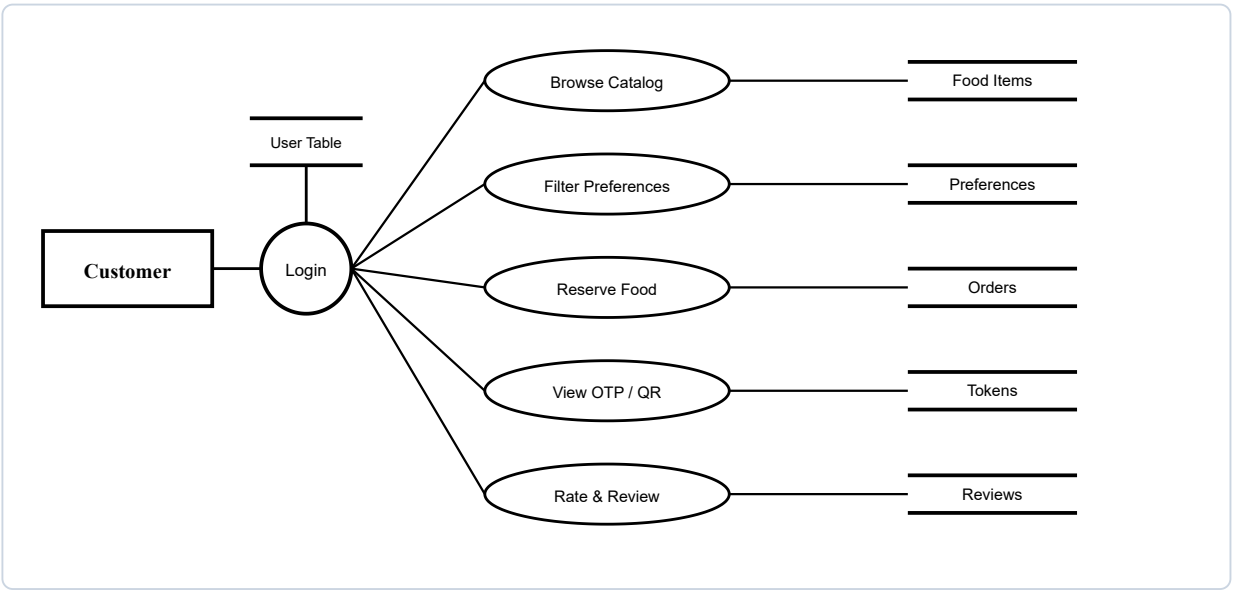
Figure 6.2: DFD Level 0 — Context Diagram

Level 1: Admin & Customer Data Flow Diagrams

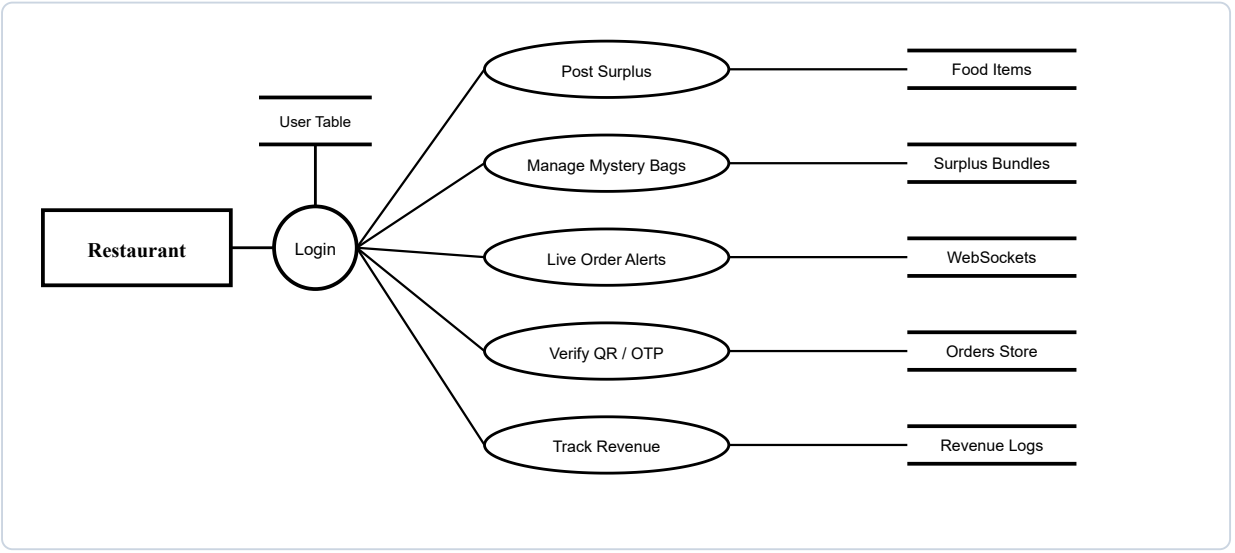
Level 1 - Admin



Level 1 - Customer



Level 1 - Restaurant Partner



Level 2: Admin Detailed Process Decomposition

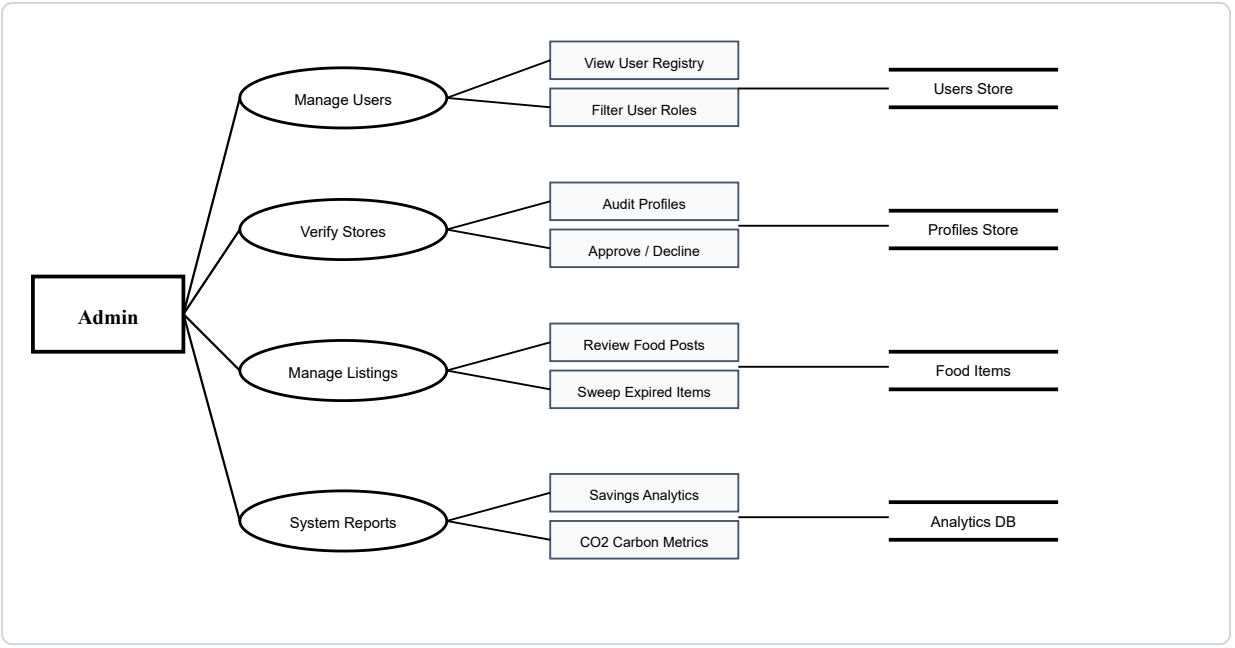


Figure 6.3: DFD Level 2 — Admin Decomposition

SYSTEM DEVELOPMENT

7. SYSTEM DEVELOPMENT

System development is a series of activities performed to design, implement, test, and deploy a software system to meet the specified requirements. The principal activities performed during the development phase can be divided into two major related sequences:

- External System Development
- Internal System Development

The major external system development activities include implementation planning, development environment setup, software installation, and cloud database provisioning. The internal system development activities involve software architecture design, database modeling, API development, frontend component engineering, authentication, authorization, testing, and security integration.

7.1 Coding

The purpose of coding is to convert the system design and requirements into executable software. It consists of a structured collection of programming statements used to implement system functions, manage data, process user requests, and establish communication between different components. Coding facilitates the development of reliable, maintainable, and efficient software by implementing the required modules and workflows.

JavaScript & Node.js

JavaScript is a high-level, dynamic, multi-paradigm programming language widely used for modern web application development. In Surplus Food Rescue & Marketplace, JavaScript powers both the interactive client-side application logic and the high-performance server-side runtime via Node.js. Node.js is an open-source, cross-platform JavaScript runtime built on Chrome's V8 engine that facilitates asynchronous, event-driven, non-blocking I/O operations, ensuring high concurrent request handling during peak food collection windows.

In Surplus Food, Node.js serves as the server runtime handling user authentication, email OTP verification, order placement pipelines, WebSocket live dispatch channels, automated cron jobs, and database CRUD operations.

React.js

React.js is an open-source JavaScript library used for building interactive and reusable user interfaces. It follows a component-based architecture, allowing large interfaces to be divided into smaller reusable components.

In Surplus Food, React.js (powered by Vite) is utilized to engineer the responsive frontend interfaces across all three user modules: Customer, Restaurant Partner, and Admin. It integrates Lucide-React icons,

HTML5 QR camera scanning modules, dynamic countdown timers, and client-side PDF document generation.

Express.js Framework

Express.js is a fast, minimalist web application framework for Node.js. It provides features for developing RESTful APIs, handling HTTP requests, managing routes, and implementing middleware. Express.js simplifies communication between the frontend and backend by providing structured API endpoints and handling HTTP methods such as GET, POST, PUT, and DELETE. It also supports request validation, authentication middleware, error handling, and JSON data processing.

MongoDB Database

MongoDB is a NoSQL, document-oriented database used for storing and managing application data. Unlike relational databases that store information in tables and rows, MongoDB stores data in flexible JSON-like documents within collections.

MongoDB Atlas is deployed in Surplus Food to store distributed documents across users, restaurant profiles, surplus food items, customer reservations, and ratings. MongoDB's flexible schema models effortlessly handle variable attributes like allergen lists, dietary tags, dynamic markdown price history, and nested payment credentials.

Standard Libraries & Middleware Packages

The Node.js development ecosystem provides several packages through Node Package Manager (npm) to support authentication, security, file management, and application configuration:

- **JWT (JSON Web Token):** Used for secure token-based authentication and role-based access control. Upon login, a signed JSON Web Token is issued containing the user's role (Customer, Restaurant, Admin), ensuring that protected API endpoints and administrative dispatch actions remain strictly safeguarded.
- **Bcrypt:** A password-hashing library used to securely hash user passwords using 10 salt rounds before storing them in the database, protecting user credentials against dictionary and rainbow table attacks.
- **Socket.io:** Powers the bidirectional, sub-second WebSocket communication layer essential for broadcasting incoming customer reservations directly to active kitchen dashboards without requiring manual page reloads.
- **Multer:** Middleware used for handling multipart/form-data, utilized for uploading restaurant banners and food item photos safely to the server uploads directory.
- **Node-Cron:** A background task scheduler that runs automated routines every minute to sweep expired listings and execute 20% dynamic markdowns during the final 30 minutes of store pickup windows.
- **Dotenv:** Used to load environment variables from a .env file, keeping sensitive credentials such as database connection URIs and JWT secrets isolated from the codebase.
- **CORS:** Cross-Origin Resource Sharing enables controlled communication between the React.js client and the Express.js server when hosted across different origins.

SYSTEM TESTING AND IMPLEMENTATION

8. SYSTEM TESTING AND IMPLEMENTATION

System testing is a crucial milestone in the engineering of Surplus Food Rescue & Marketplace. Rigorous testing verifies that all modules operate with extreme reliability, fail-safe resilience, low latency, and zero data loss under concurrent consumer and restaurant transaction loads.

Software testing is an important part of software quality assurance. It involves the systematic evaluation of the system's requirements, design, coding, functionality, and user interaction. Testing is performed throughout the development process to verify that the implemented system produces the expected results under specified conditions.

8.1 Types of Testing

The different types of testing performed for the Surplus Food platform include Unit Testing, Integration Testing, System Testing, Validation Testing, and User Acceptance Testing:

Unit Testing

Unit testing involves testing individual modules or components of the application separately to ensure that each component performs its intended function correctly. It is generally performed during the coding stage to identify errors early.

In Surplus Food, unit testing was conducted on individual software units including password hashing, 6-digit OTP token generation, JWT signing, price validation (ensuring offer price is strictly less than original price), quantity checks, and countdown calculations. Each function was validated against boundary cases and expected numerical outputs.

Integration Testing

Integration testing involves combining individually tested modules and checking whether they work correctly together. It focuses on identifying errors in communication and data exchange between different components of the system.

In Surplus Food, integration testing was performed across cooperating pipelines such as:

- User registration, OTP verification, and JWT session authorization pipelines.
- Restaurant surplus food creation and immediate reflection on the customer catalog.
- Customer booking checkout triggering real-time WebSocket notifications to the merchant.
- Token / QR validation decrementing restaurant available stock and updating reservation status.
- React.js frontend, Express REST APIs, Socket.io gateway, and MongoDB Atlas database.

System Testing

System testing evaluated the complete Surplus Food platform as a unified marketplace. The platform was evaluated under high concurrency to simulate evening rush-hour food collections, verifying sub-second WebSocket broadcast latencies, accurate countdown timers, and resilience against network timeouts.

The validation suite confirmed proper role isolation: customers cannot access merchant management panels or admin audit tools, and restaurants can only access and modify their own inventory catalogs.

Validation Testing

Validation testing ensures that the developed system satisfies all functional requirements and operational expectations established during the analysis and design stages. It verifies that the application correctly fulfills all mission workflows for the Customer, Restaurant Partner, and Admin.

Validation testing includes checking:

- Correct user registration and instant 6-digit email OTP authentication.
- Strict role-based authorization preventing unauthorized data access.
- Accurate surplus item listing, image uploading, and Mystery Bag configuration.
- Accurate search filtering across cuisines, maximum price bounds, and allergen tags.
- Reliable order checkout, OTP token issuance, and QR code rendering.
- Real-time WebSocket alerts upon incoming customer bookings.
- Automated node-cron background sweeps of expired food items.
- Administrative restaurant audit approvals and platform listings sweeps.

User Acceptance Testing (UAT)

User Acceptance Testing (UAT) was conducted by simulating real-world bakery and restaurant closing scenarios with test users acting as Customers, Restaurant Kitchen Staff, and Administrators. Feedback verified that reserving surplus food and redeeming passes via 6-digit OTP tokens was exceptionally straightforward and rapid.

The major areas considered during user acceptance testing include:

- Output screen design and aesthetic visual presentation.
- Input form ergonomics and validation feedback.
- Navigation and menu structure.
- QR scanner responsiveness on mobile devices.
- Checkout and token retrieval flow.
- Dashboard functionality and metrics clarity.

8.2 Implementation

Implementation is the vital phase where the software design specifications of Surplus Food were translated into an operational, production-ready marketplace platform. The implementation integrated the React.js client interface with the Node.js / Express.js backend, configured WebSocket channels, seeded initial sample test accounts, and deployed the application.

The implementation process includes:

- Careful planning of the system deployment.
- Setting up the React.js, Node.js, Express.js, and MongoDB environment.
- Configuring the database connection strings and environment security variables.
- Integrating frontend interfaces with backend RESTful APIs.
- Implementing authentication and role-based access control middleware.
- Testing all modules and fixing identified errors.
- Preparing user guidance and documentation.

SYSTEM MAINTENANCE

9. SYSTEM MAINTENANCE

Maintenance is the process of making necessary changes and improvements to a software system after its implementation. These changes may include fixing errors, adapting the system to changes in the software environment, improving performance, enhancing security, and adding new features. When changes are implemented during the maintenance phase, the affected parts of the system must be systematically reviewed and tested to maintain system stability and data integrity.

There are three types of maintenance:

- **Corrective Maintenance** – Fixing bugs, errors, and system failures.
- **Adaptive Maintenance** – Updating the system to support changes in software, hardware, or operating environments.
- **Perfective Maintenance** – Improving existing features and adding enhancements based on user requirements.

Maintenance is an essential component of the software development lifecycle, particularly for an active consumer marketplace like Surplus Food. Ongoing maintenance ensures 99.99% operational uptime, security patching against newly discovered vulnerabilities, optimization of database queries, and continuous calibration of automated price markdown algorithms.

9.1 Key Maintenance Activities & Importance

Key maintenance activities for Surplus Food encompass: (1) Problem Identification & Error Monitoring through server health logs, WebSocket connection states, and API latency trackers; (2) Design & Code Refinement across React components, Express controllers, and Mongoose schemas; (3) Regression Testing of booking and verification pipelines; and (4) Configuration Management via GitHub and automated deployment pipelines.

1. Problem Identification & Analysis

Monitoring server errors, API responses, database issues, and user feedback to identify and analyze problems related to login, profiles, reservations, notifications, or other system functions.

2. Design & Code Modification

Modifying React.js components, Express.js APIs, Node.js backend services, or MongoDB models to fix issues and introduce improvements without affecting existing system functionality.

3. Regression Testing & Verification

Re-testing important application functions such as authentication, role-based access, surplus food reservation, WebSocket dispatch alerts, and camera QR verification to ensure that new code updates do not compromise system stability.

4. Deployment & Configuration Management

Deploying updated application versions, managing environment variables and configuration settings, maintaining database configurations, and using GitHub for version control and tracking system changes.

FUTURE ENHANCEMENT

10. FUTURE ENHANCEMENTS

The Surplus Food Rescue & Marketplace platform has established a high-performance foundation for urban food rescue. To further expand its sustainability impact and adapt to growing commercial demands, several cutting-edge technological enhancements are planned for future iterative releases:

10.1 AI-Powered Surplus Waste Prediction

Integrating machine learning predictive models (e.g. Random Forest regression, LSTM neural networks) trained on historical restaurant sales, weather conditions, holidays, and local foot traffic. This engine will automatically suggest optimized preparation quantities to restaurant chefs each morning, preventing surplus from being generated in the first place.

10.2 Native Mobile Applications (iOS & Android)

Developing cross-platform mobile apps using React Native or Flutter, offering geofenced push notifications that alert consumers on their smartphones whenever their favorite bakery publishes surprise bags within a 2-kilometer radius.

10.3 Direct Integration with Commercial POS Systems

Creating automated integration plugins for popular Point-of-Sale (POS) systems (such as Toast, Square, and Clover). When items remain unsold in the restaurant register 45 minutes prior to closing, the POS integration can automatically publish them to the Surplus Food marketplace without requiring manual intervention from kitchen staff.

10.4 Dedicated NGO & Charity Donation Routing

Introducing an automated charity dispatch pipeline. For items that remain unreserved by retail consumers 15 minutes before closing, the platform can automatically dispatch notifications to local food banks, shelters, and voluntary relief organizations for free bulk pickup.

CONCLUSION

11. CONCLUSION

Urban food wastage represents an immense economic loss and ecological hazard that demands innovative, technological intervention. While billions of dollars worth of fresh, wholesome meals are discarded each day, millions of consumers seek affordable dining options amid soaring food inflation. Surplus Food Rescue & Marketplace was conceived, engineered, and evaluated specifically to eliminate this systemic divide.

By harmonizing food businesses, neighborhood consumers, and system administrators into an integrated real-time marketplace, the platform turns food waste into an opportunity for economic recovery and environmental conservation. The integration of role-based security, instant Surprise Bag reservations, automated OTP and QR redemption tokens, live WebSocket order dispatching, and background cron markdown routines transforms food rescue into a seamless, profitable, and dignified daily experience.

Developed using the modern MERN Stack (React.js, Node.js, Express.js, and MongoDB Atlas) paired with Socket.io, the project demonstrates the successful realization of an enterprise-grade full-stack web application. Surplus Food satisfies all technical, operational, and academic criteria prescribed by the APJ Abdul Kalam Kerala Technological University for the Master of Computer Applications curriculum, while establishing a meaningful and sustainable contribution to modern ecological informatics.

APPENDIX

12. APPENDIX

Figure 12.1: Hero Landing Page & Sustainability Mission Portal



Figure 12.2: Multi-Role Authentication & Email OTP Verification

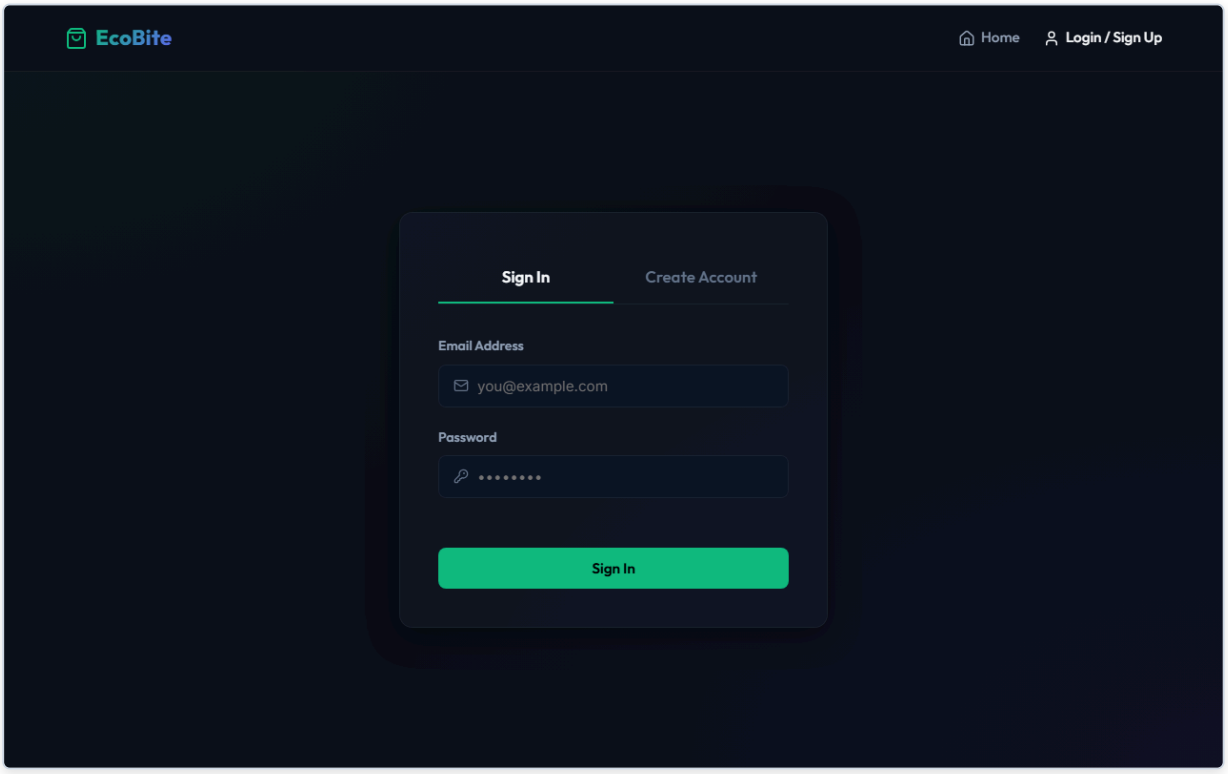


Figure 12.3: Customer Surplus Food Exploration Catalog & Surprise Bags

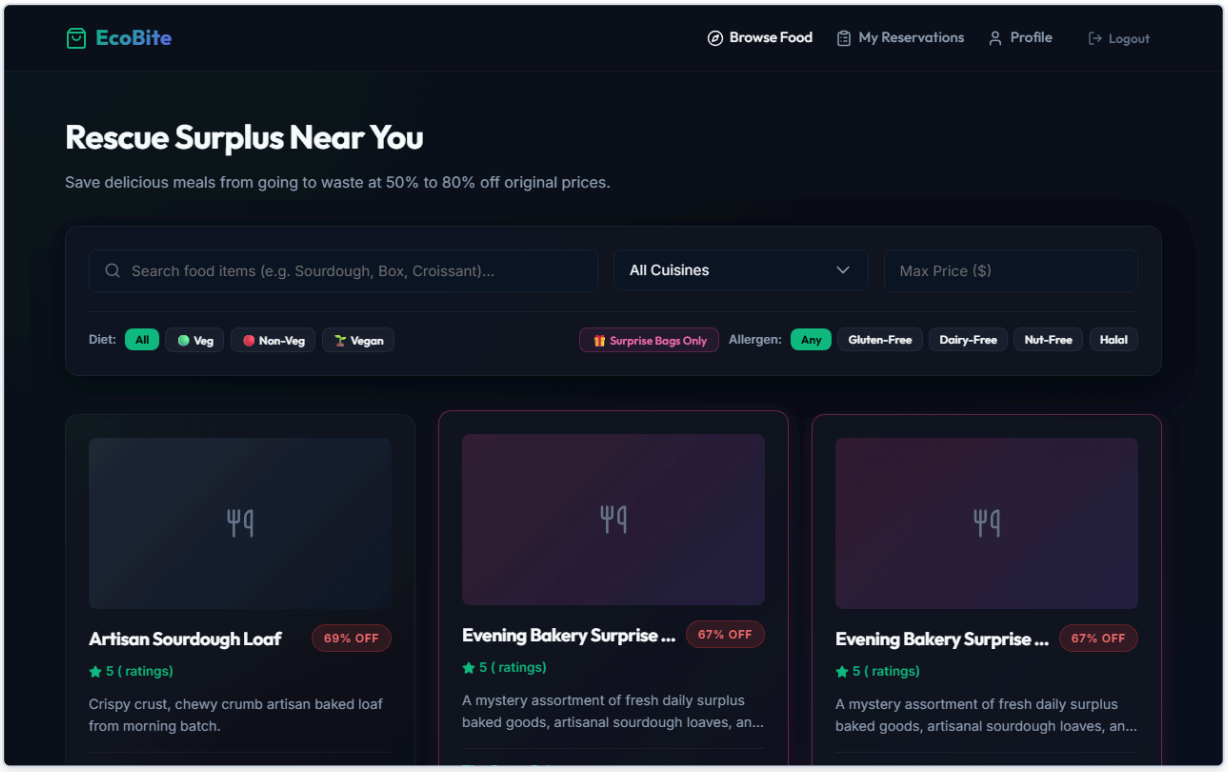


Figure 12.4: Customer Active Reservations, OTP Token & QR Collection Pass

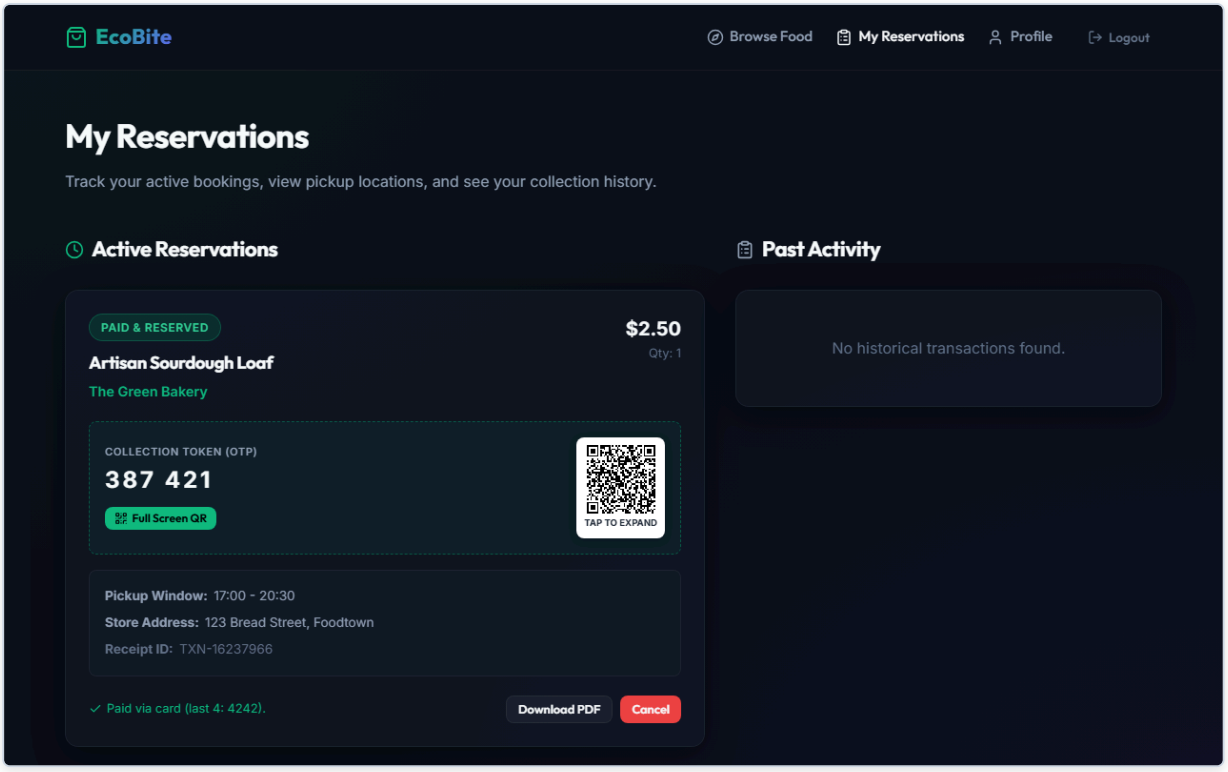


Figure 12.5: Customer Profile & Account Settings Management

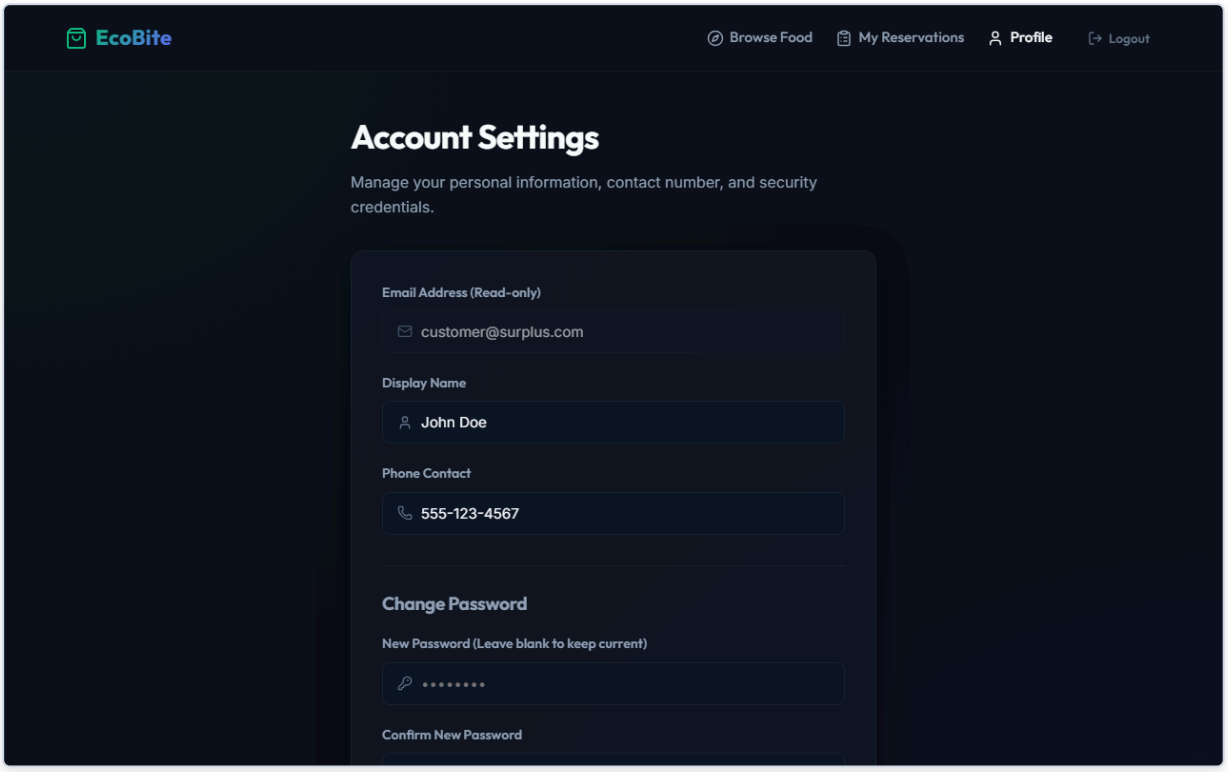


Figure 12.6: Restaurant Business Hub & Live Surplus Inventory Management

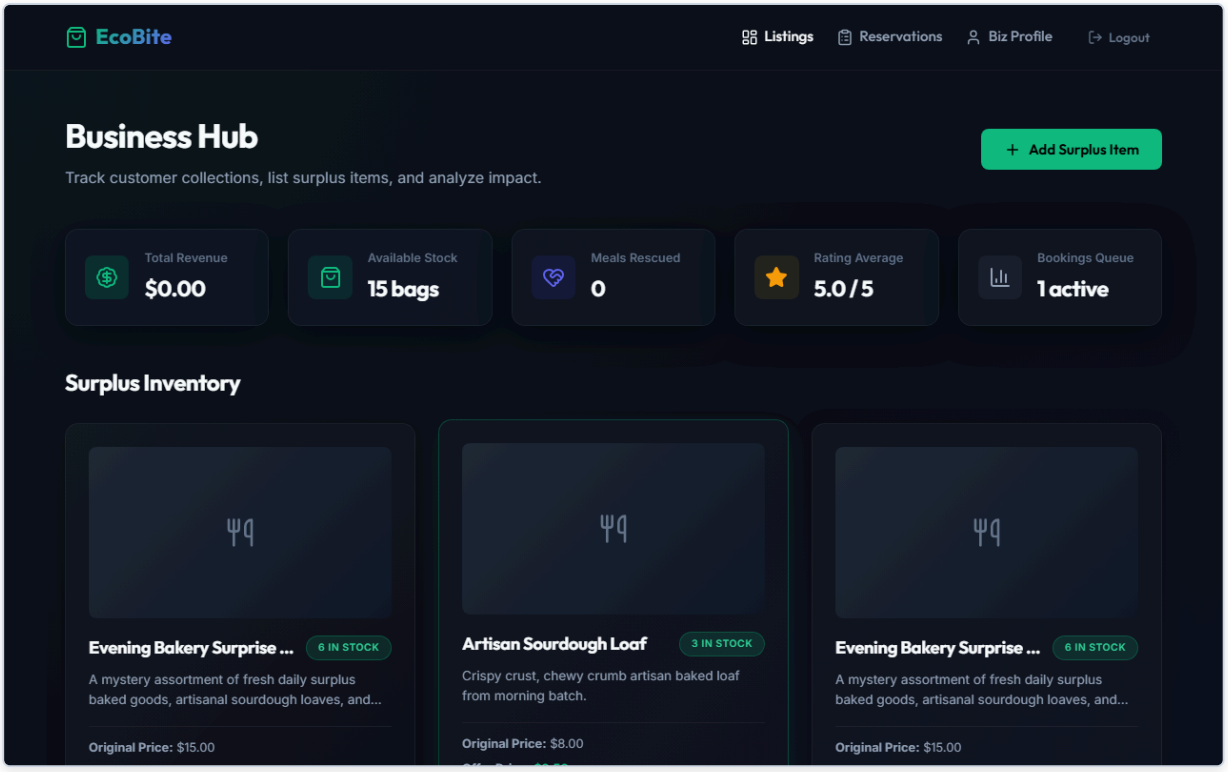


Figure 12.7: Restaurant Customer Bookings Queue & QR Scanner Verification

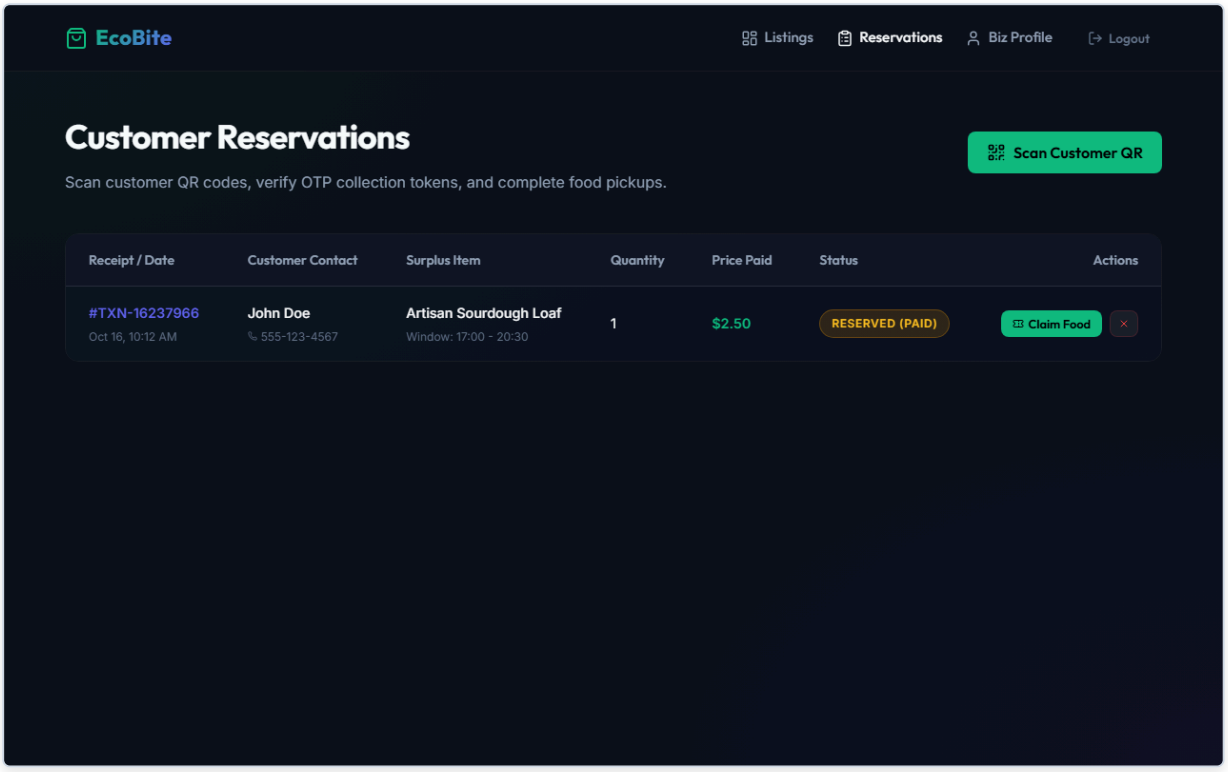


Figure 12.8: Admin Command Center & Environmental Impact Analytics

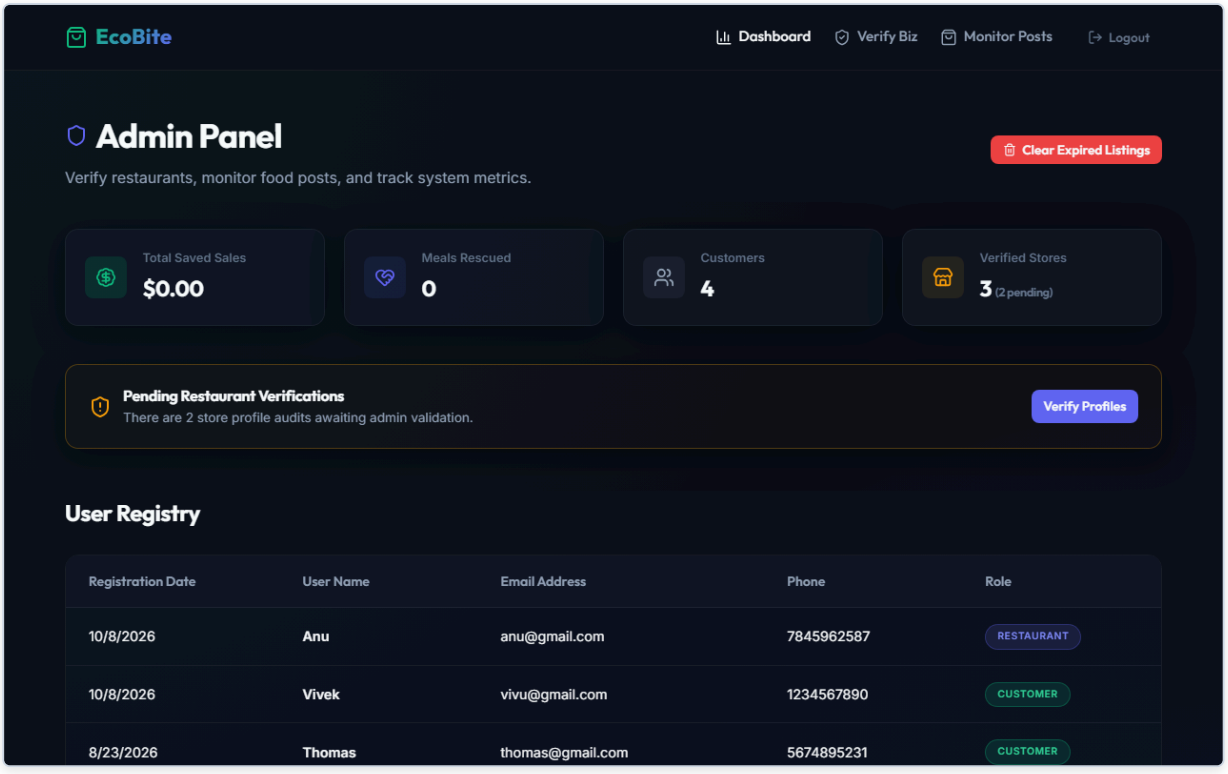


Figure 12.9: Admin Restaurant Verification & Profile Audit Console

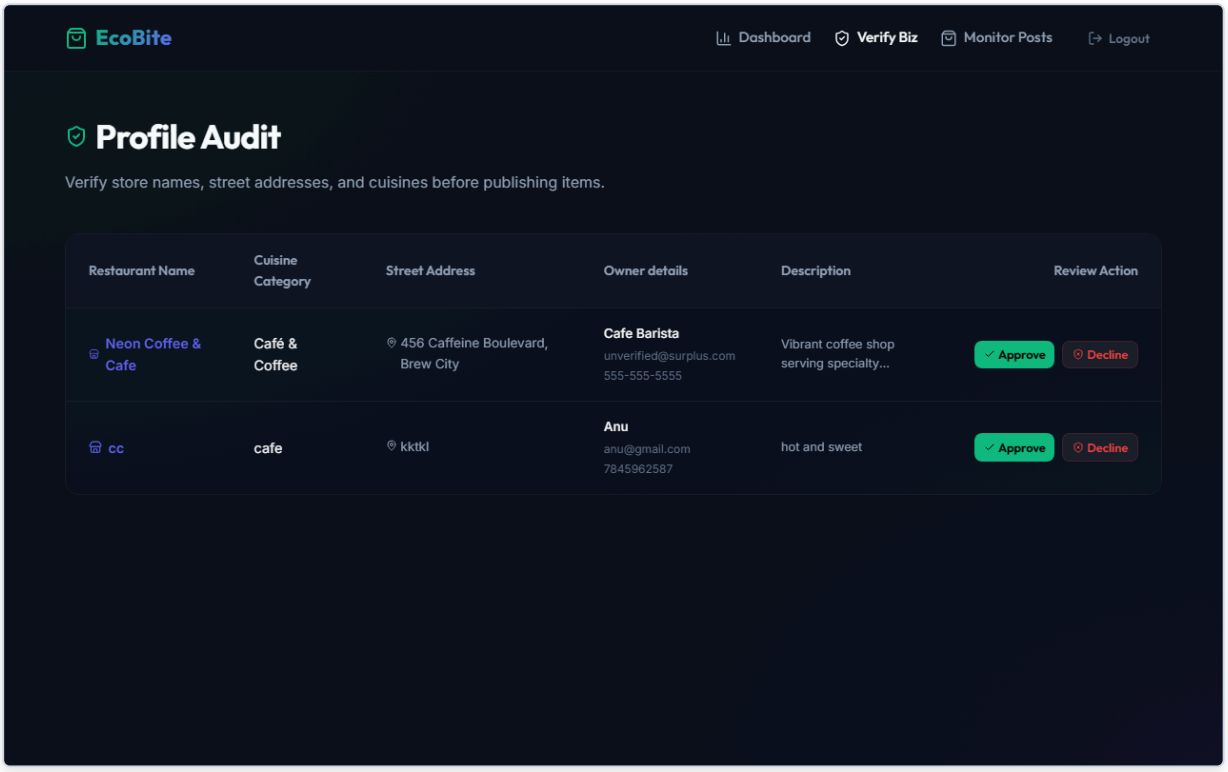
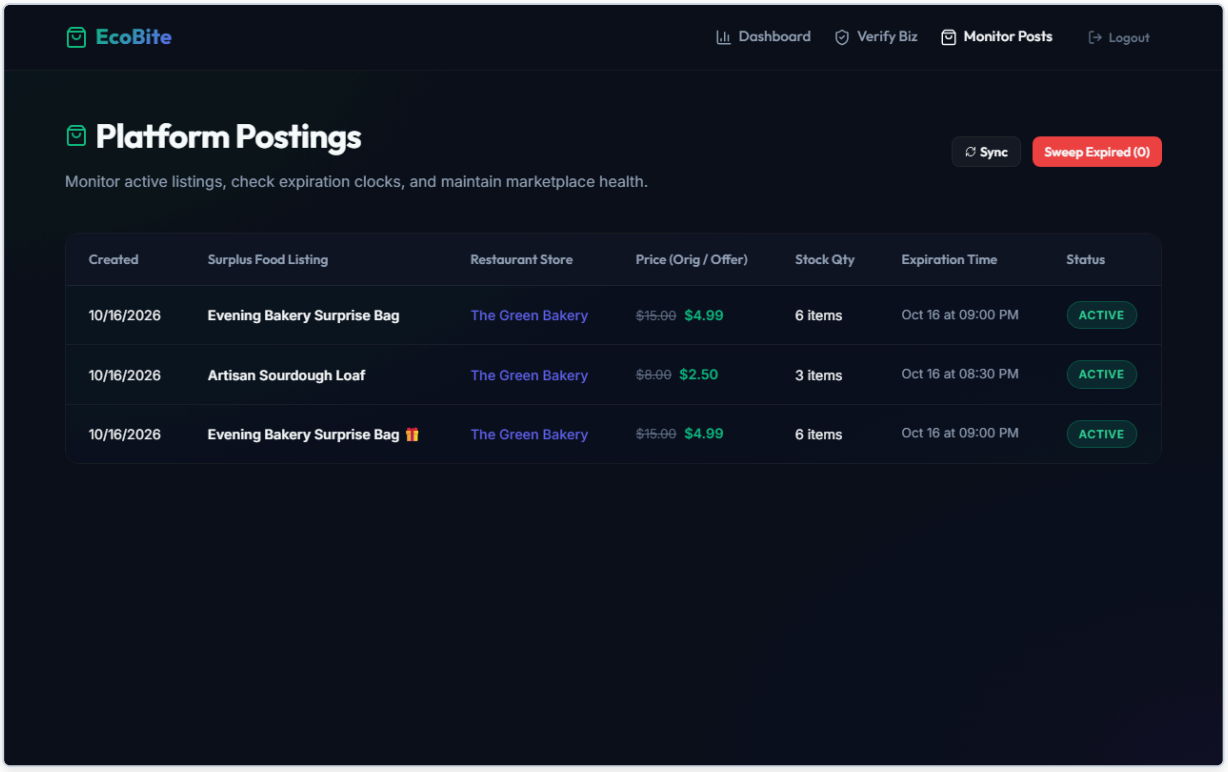


Figure 12.10: Admin Global Platform Postings & Expired Items Sweeper



BIBLIOGRAPHY

13. BIBLIOGRAPHY

Websites

[1] https://react.dev/

[2] https://nodejs.org/docs/latest/api/

[3] https://expressjs.com/

[4] https://www.mongodb.com/docs/

[5] https://mongoosejs.com/docs/

[6] https://socket.io/docs/v4/

[7] https://jwt.io/introduction

[8] https://vitejs.dev/

[9] https://lucide.dev/

[10] https://www.npmjs.com/package/node-cron

Books

[1] David Flanagan, *JavaScript: The Definitive Guide*, 7th Edition, O'Reilly Media, 2020.

[2] Ethan Brown, *Web Development with Node and Express: Leveraging the JavaScript Stack*, 2nd Edition, O'Reilly Media, 2019.

[3] Alex Banks and Eve Porcello, *Learning React: Modern Patterns for Developing React Applications*, 2nd Edition, O'Reilly Media, 2020.

[4] Kristina Chodorow, *MongoDB: The Definitive Guide: Powerful and Scalable Storage*, 3rd Edition, O'Reilly Media, 2019.

[5] Rohit Rai, *Socket.IO Real-Time Web Application Development*, Packt Publishing, 2017.